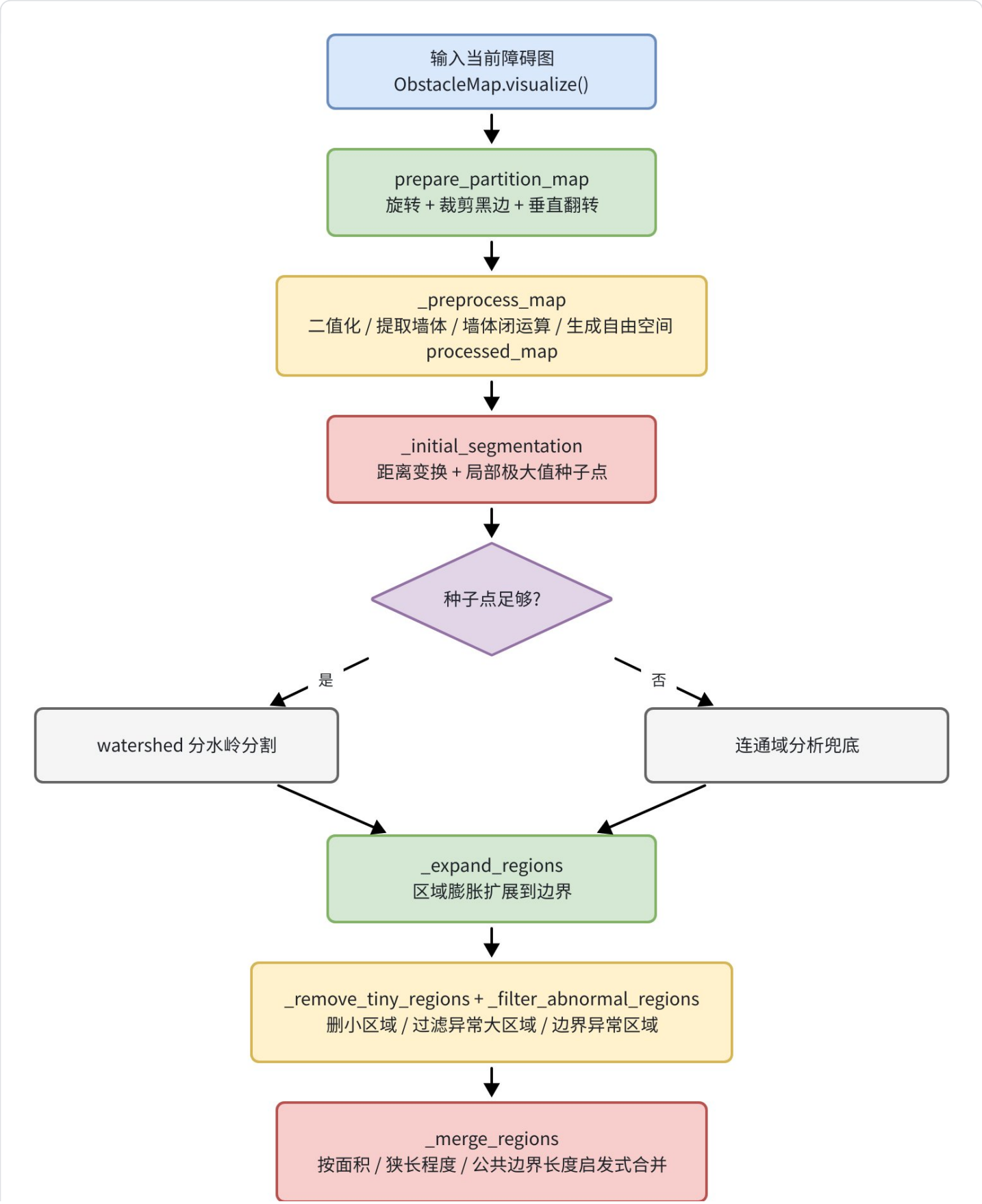
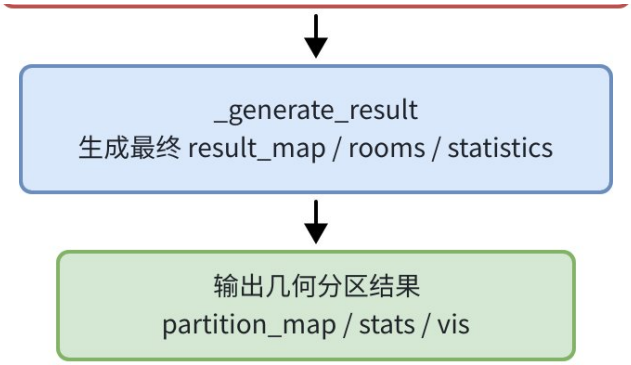
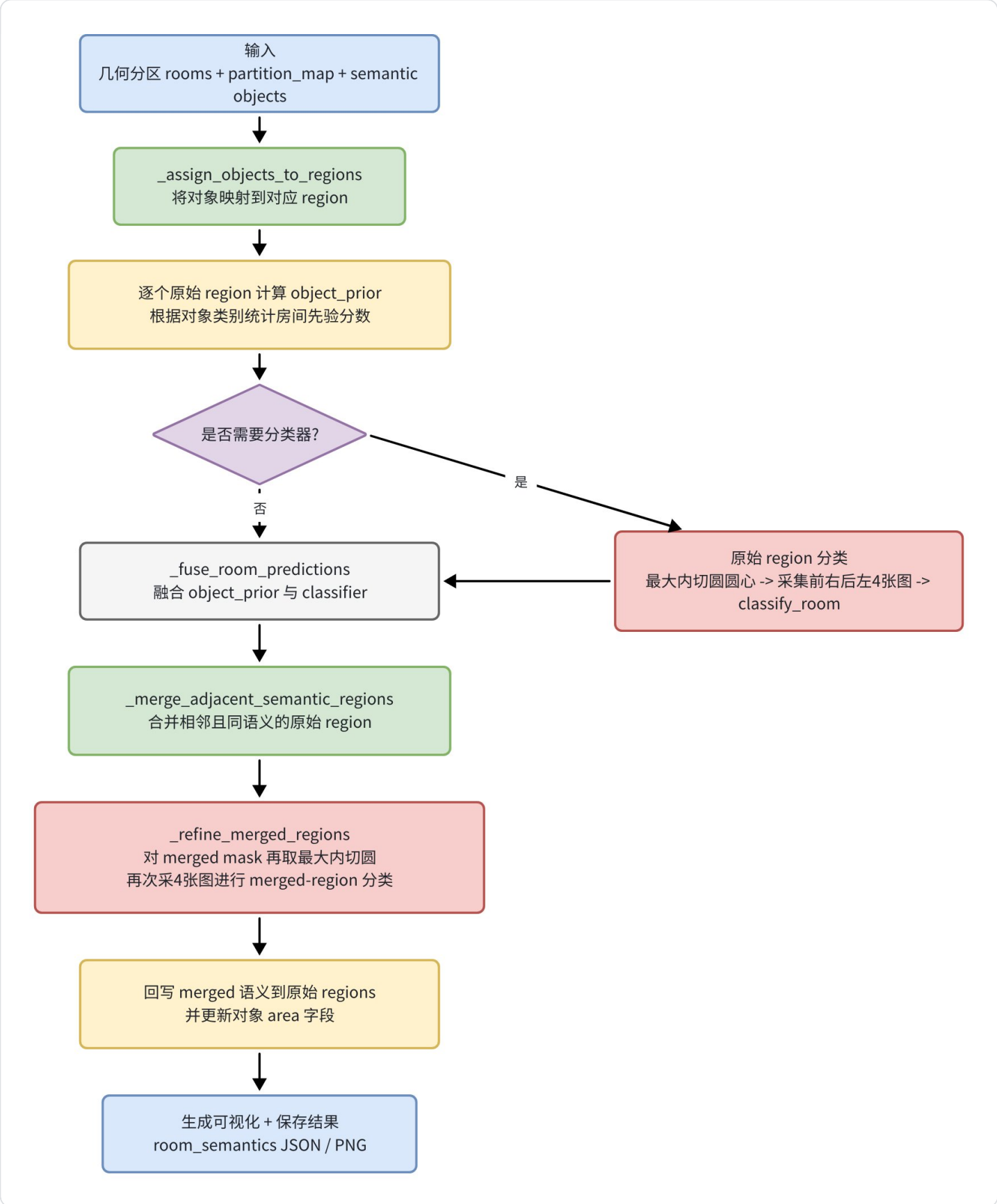
地图分区及房间语义地图演进

一、当前算法流程




_generate_result
生成最终 result_map / rooms / statistics
输出几何分区结果
partition_map / stats / vis

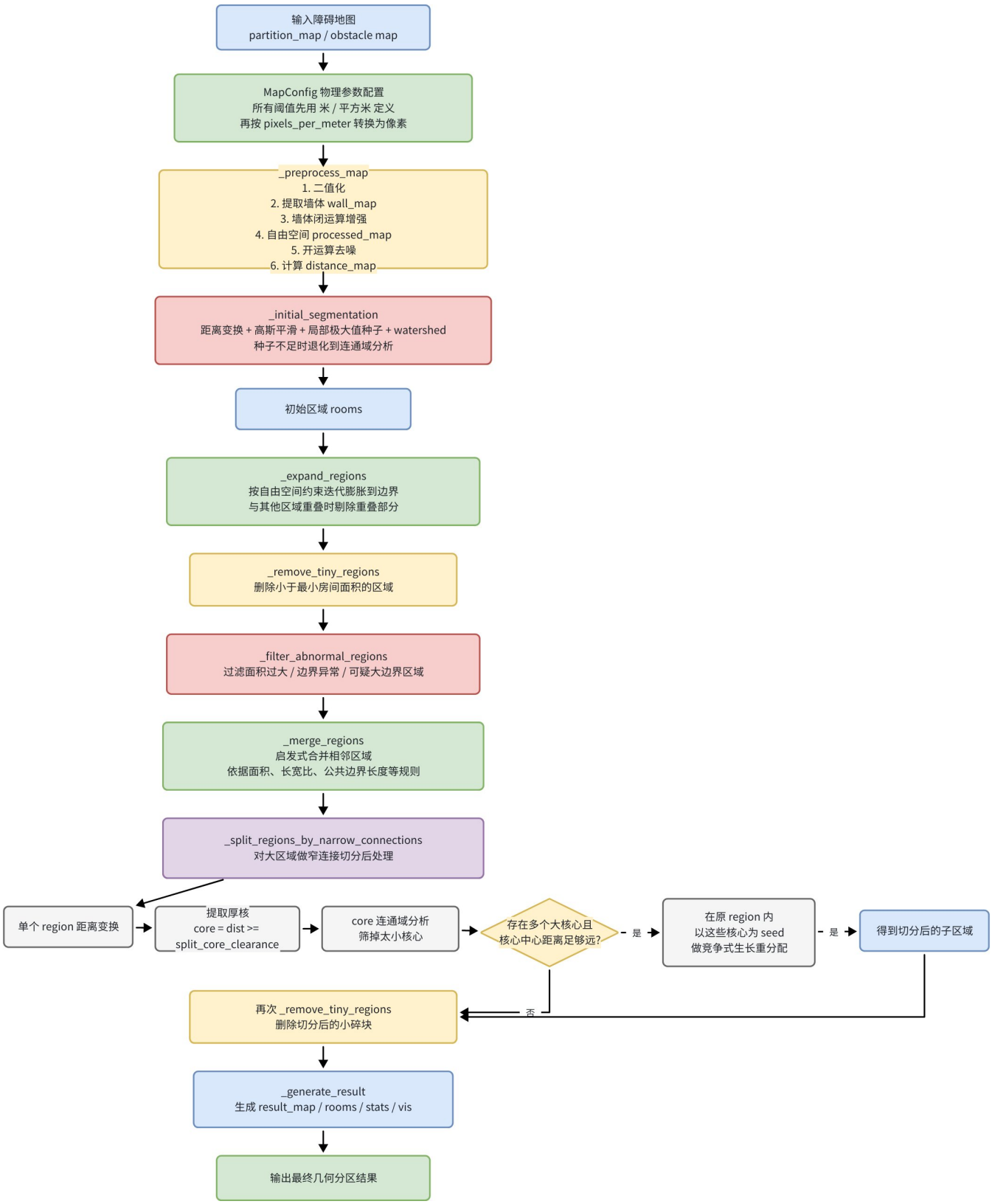
分区算法



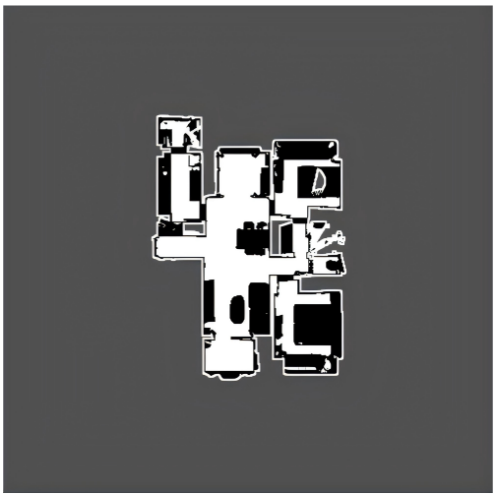
房间语义

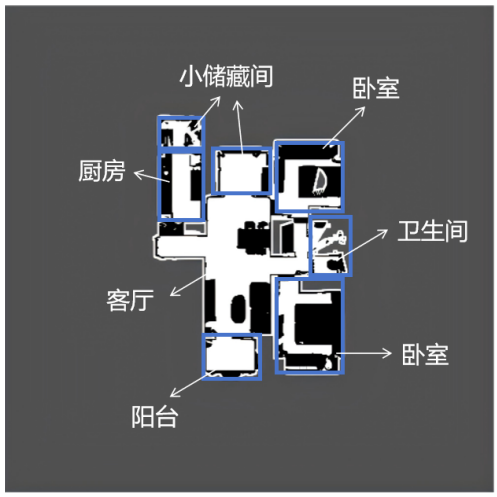
二、基于当前算法框架优化

针对走廊等狭长区域与房间相邻时出现区域无法区分

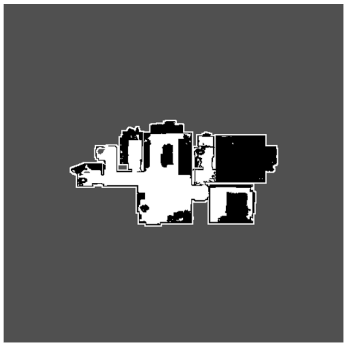


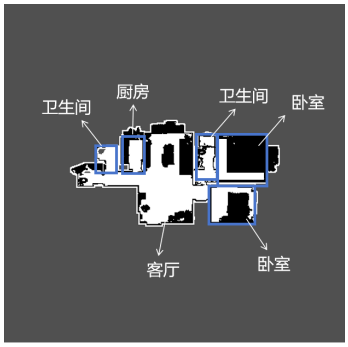
修改后效果如下，但仍存在对于不同场景效果不稳定的情况：

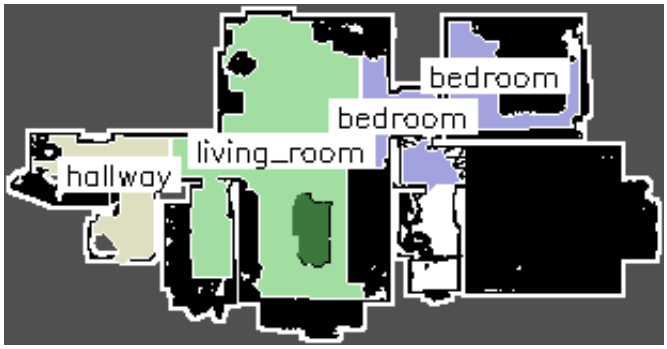


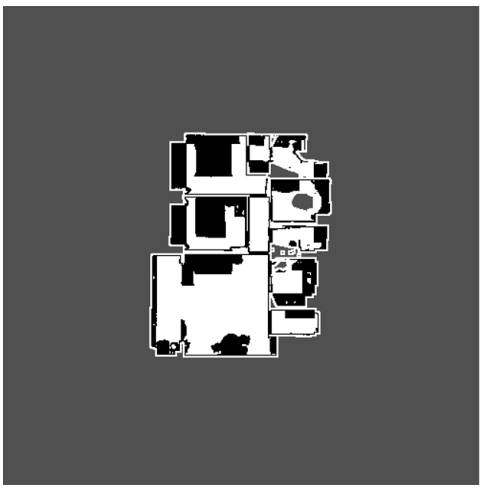

小储藏间
卧室
厨房
卫生间
客厅
卧室
阳台

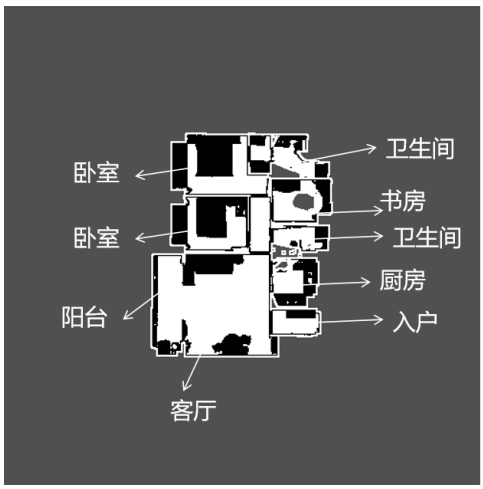
bedroom
kitchen
bedroom
hallway
living_room
balcony



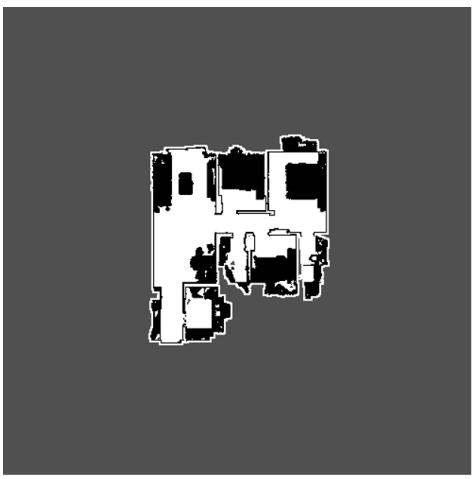

卫生间
厨房
卫生间
卧室
客厅
卧室

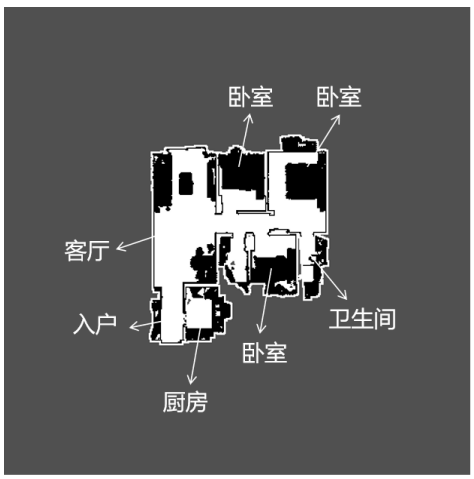

bedroom
bedroom
living_room
hallway



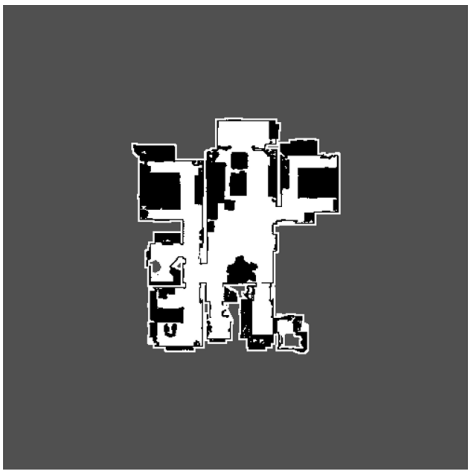

卫生间
卧室
书房
卧室
卫生间
厨房
阳台
入户
客厅

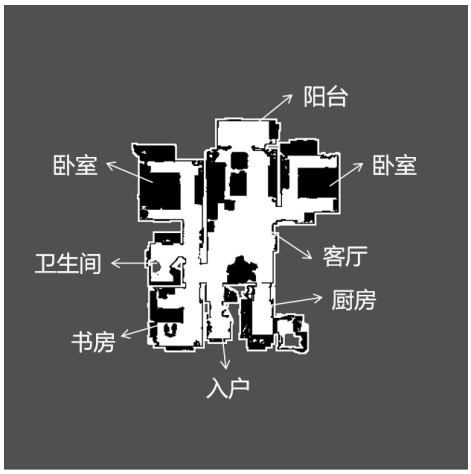
bedroom
hallway
bedroom
laundry
kitchen
living_room
laundry

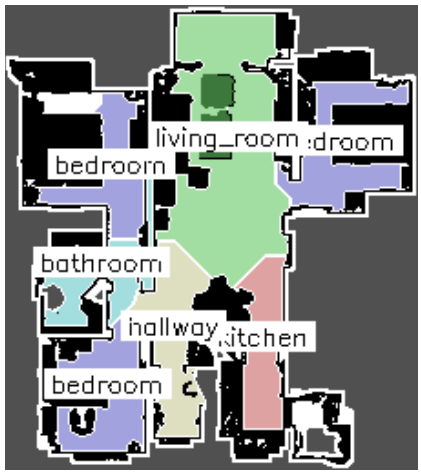



卧室
卧室
客厅
入户
卫生间
卧室
厨房

bedroom
living_room
unknown
bathroom
hallway



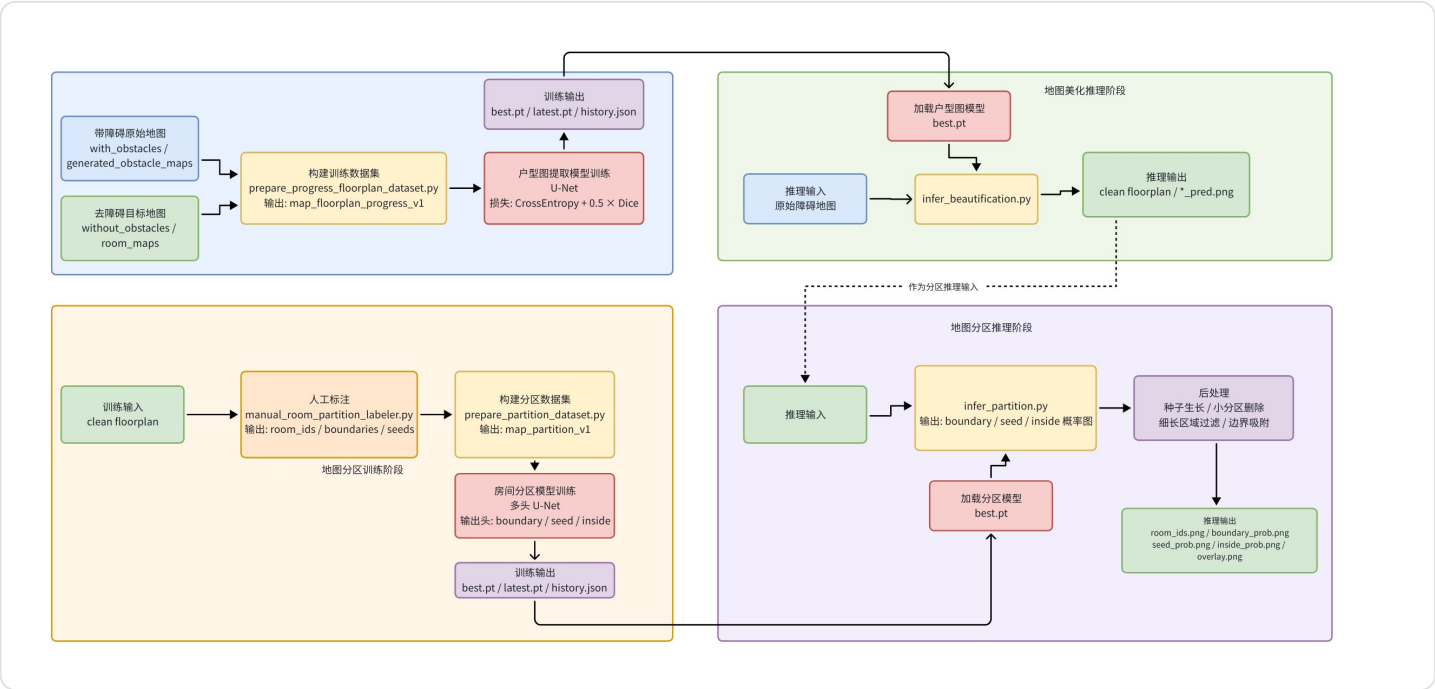




三、基于深度学习实现通用地图美化、分区调研

3.1 当前实现流程图

当前流程先利用带障碍地图和无障碍目标图训练地图美化模型得到美化后地图，再基于人工标注的房间分区数据训练多头 U-Net 预测房间边界、房间核，最后通过后处理生成带房间实例编号的分区结果图。



3.2 输入输出说明

3.2.1 地图美化

数据输入：

> 原始地图-基于ASM构建过程提取0.2m-2m高度区间的地图

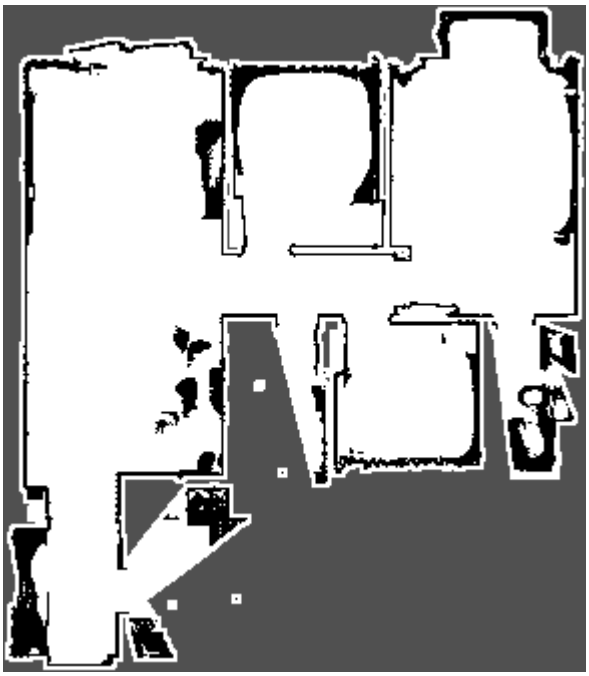
标签设置：

- 墙体或障碍边界：在“美化后的户型图”里，黑色只应该表示结构墙体
- 室外背景：也就是房屋外部、未进入的区域
- 室内可用空间：也就是去掉家具和杂点之后的人可通行区域

损失函数学习对应目标各部分mask，从而将输入的带障碍图修改为去除障碍图像：

- 墙体 mask：without_obstacles == 0
- 室外 mask：without_obstacles == 80
- 室内 mask：without_obstacles == 255
- 障碍物 mask：(with_obstacles == 0) AND (without_obstacles != 0)
- 恢复室内区域 mask：(with_obstacles == 0) AND (without_obstacles == 255)

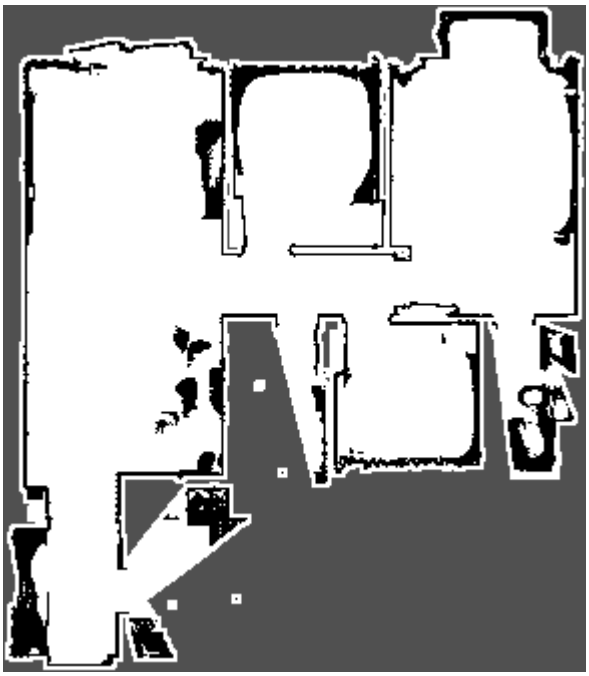
推理输出：



3.2.2 地图分区

数据输入：

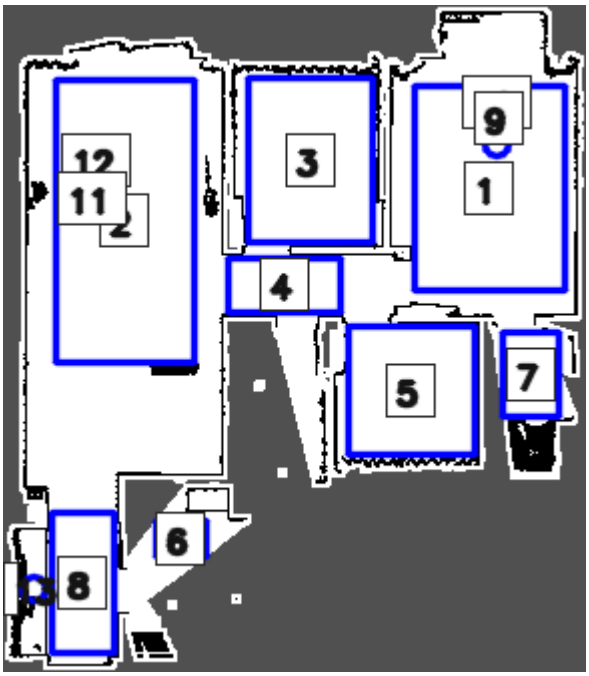
> 训练数据来源-地图美化过后的户型图



标签设置：

- inside：标签表示室内可分区区域，由 clean floorplan 中值为 255 的像素生成。
- boundary：标签表示不同房间之间的分界线，由 room_ids 图中相邻不同房间编号的交界位置计算得到。
- seed：标签表示每个房间的中心区域，由 room_ids 对每个房间掩码进行形态学腐蚀后生成，用于后处理中房间区域的生长起点。

模型推理输出：



3.3 模型训练占用及效率

地图美化部分：

- 单张推理时间：0.169s
- 推理占用（cpu）：748.45 MB

地图分区部分：

- 单张推理时间：2.408s
- 推理占用（cpu）：776.05MB

3.4 模型优点与缺点

优点：

1. 当前两阶段模型都基于 `U-Net` 结构实现，代码依赖简单，PyTorch 环境即可运行。
2. 输入并不是复杂的 RGB 传感器数据，而是统一编码后的地图数据，因此在不同地图来源之间迁移较方便。
3. 数据准备、训练、推理都已经拆成独立脚本，目录结构清晰，迁移到其他项目时改动面相对较小。

缺点：

1. 当前实现仍依赖固定的像素语义约定，如果换一套地图编码规则，需要先重写数据预处理。
2. 分区阶段较依赖当前手工构建的数据组织方式，例如 `room_ids / boundaries / seeds / inside_masks`，迁移时需要保证目标项目也能提供同样格式。
3. 一些推理后处理参数和当前地图尺度强相关，换到不同尺寸或不同比例尺的地图时，需要重新调整。
4. 当模型在新数据上表现不稳定时，最直接的提升方式仍然是继续补标，这意味着系统在短期内仍是“人工驱动型”迭代。

3.5 总结：

整体看当前方法仍是传统几何方法的“变种”，虽在过程中不再依赖纯几何方法的计算，但输入部分仍是传统几何方法的地图结构，所以也无法摆脱换个场景就需要重新进行适配的问题，但优点是只需要前端地图形式确定，通过仿真环境即可初步构建数据集进行训练得到初代模型并进行后续迭代，效果将稳步提升。

四、去几何地图化分区形式构思

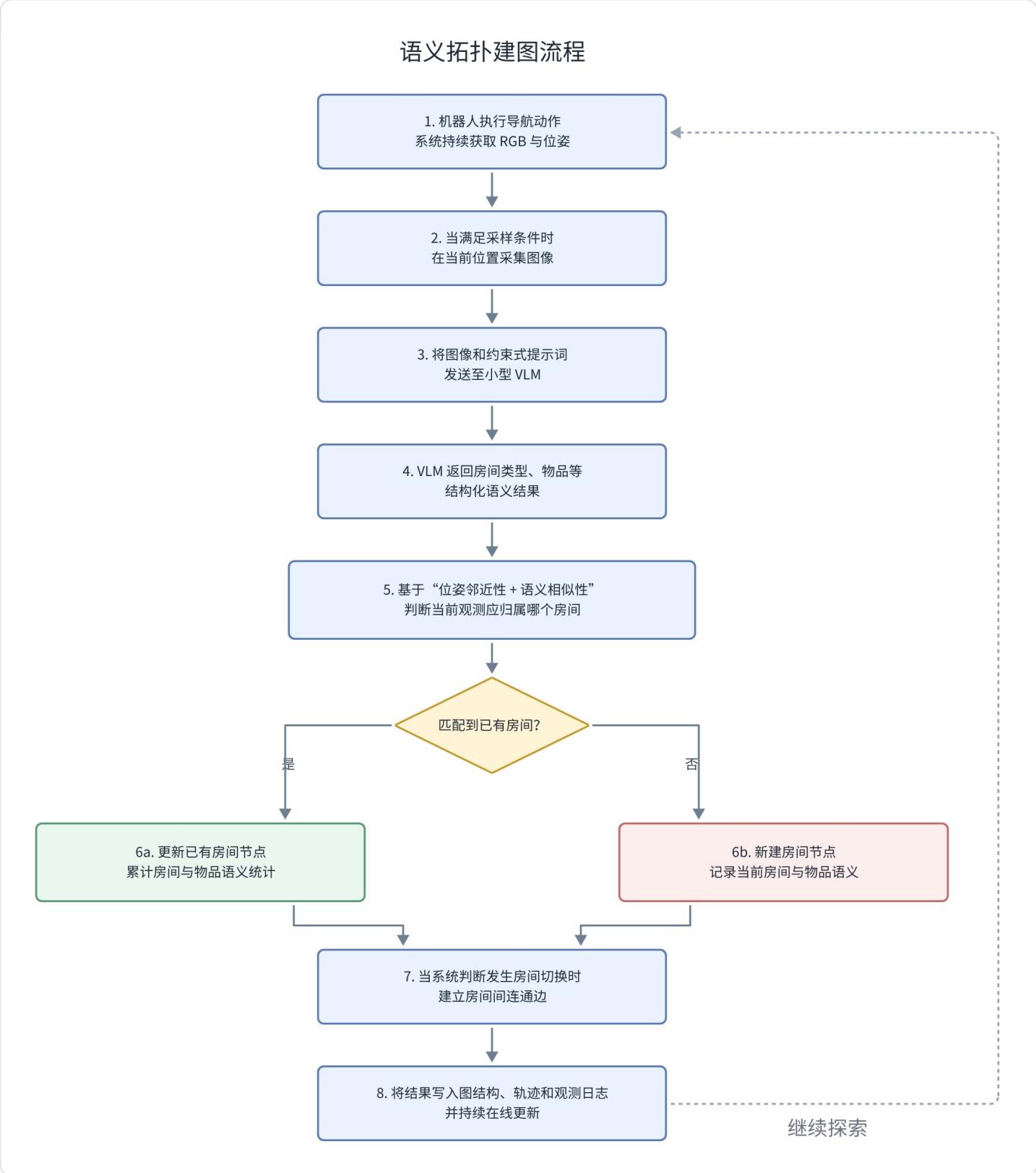
4.1 实现方案说明

与传统的度量地图或三维重建方案不同，本方案关注的是高层语义结构，核心输出不是稠密点云或精确网格，而是相较稀疏的房间及物品语义的拓扑关系，主要包含：

- 房间级语义节点
- 房间间连通关系
- 房间内显著物体语义
- 房间内物品位置

工作流程

系统按一定频率采集当前观测与粗定位信息，并结合历史观测和运动轨迹送入 VLM 做房间语义判断，再将结果在线聚合为房间节点、物体语义和房间连通关系，逐步构建完整的语义拓扑图。

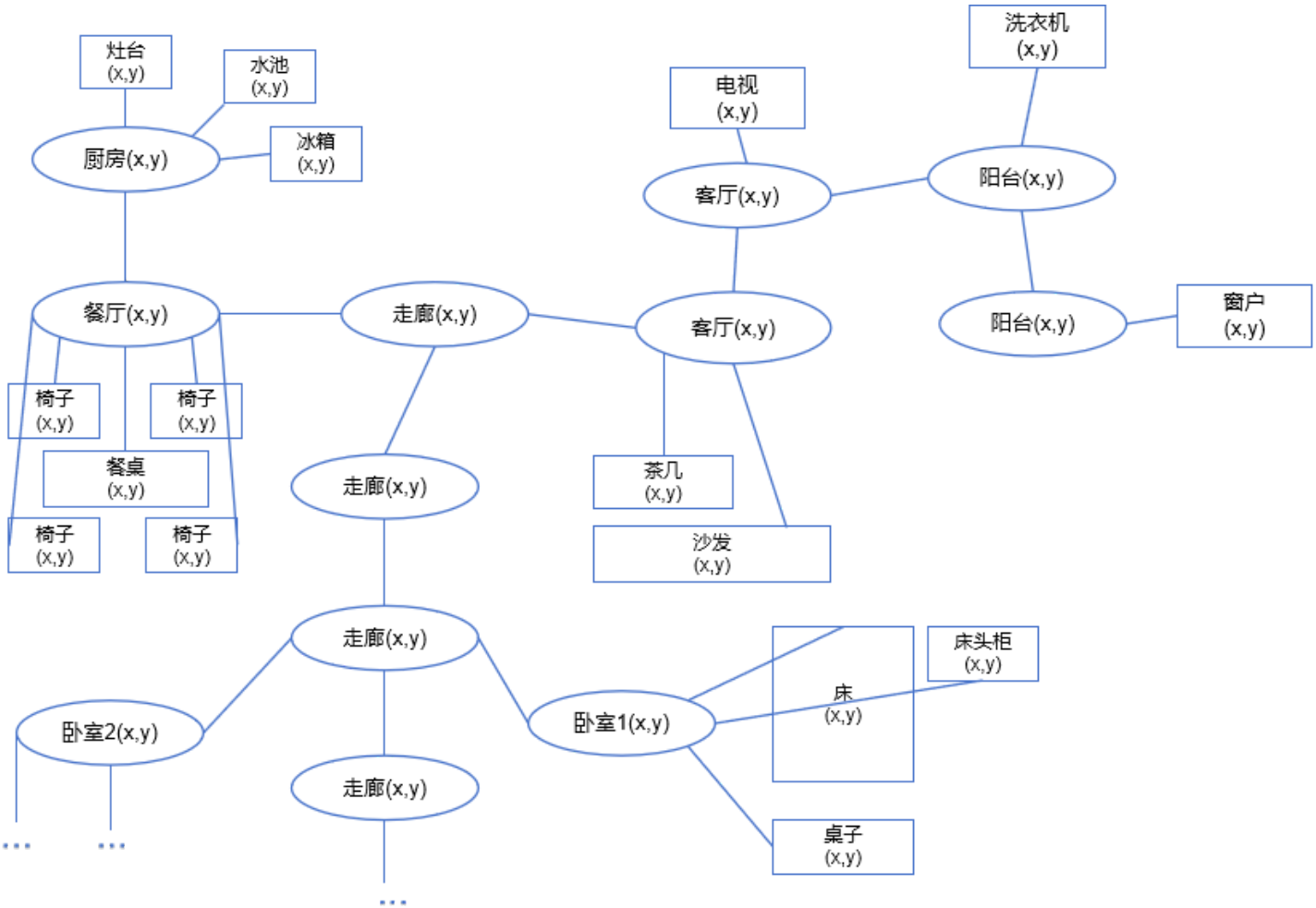


4.2 输入输出说明

数据输入: RGB数据+定位数据（绝对位置精度要求低，相对位置精度要求稍高一些）

数据输出：

拓扑图示意图：



真实保存形式：**json**

代码块

```
{
  "scene_id": "home_demo",
  "rooms": [
    {
      "id": "room_001",
      "room_type": "room_a",
      "position": { "x": 1.2, "y": 8.4 },
      "objects": [
        { "name": "object_001", "position": { "x": 1.0, "y": 8.9 } },
        { "name": "object_002", "position": { "x": 1.8, "y": 8.7 } },
        { "name": "object_003", "position": { "x": 2.0, "y": 8.2 } }
      ],
      "neighbors": ["room_002"]
    },
    {
      "id": "room_002",
      "room_type": "room_b",
      "position": { "x": 1.2, "y": 6.5 },
      "objects": [
        { "name": "object_004", "position": { "x": 0.6, "y": 5.7 } },
```

```
          { "name": "object_005", "position": { "x": 1.0, "y": 4.9 } },
          { "name": "object_006", "position": { "x": 1.9, "y": 4.9 } },
          { "name": "object_007", "position": { "x": 1.8, "y": 5.7 } }
        ],
        "neighbors": ["room_001", "room_003"]
      },
      {
        "id": "room_003",
        "room_type": "room_c",
        "position": { "x": 4.0, "y": 6.5 },
        "objects": [],
        "neighbors": ["room_002", "room_004", "room_009"]
      },
      {
        "id": "room_004",
        "room_type": "room_d",
        "position": { "x": 3.6, "y": 4.4 },
        "objects": [],
        "neighbors": ["room_003", "room_005"]
      },
      {
        "id": "room_005",
        "room_type": "room_e",
        "position": { "x": 3.5, "y": 2.5 },
        "objects": [],
        "neighbors": ["room_004", "room_006", "room_007", "room_008"]
      },
      {
        "id": "room_006",
        "room_type": "room_f",
        "position": { "x": 1.2, "y": 1.4 },
        "objects": [],
        "neighbors": ["room_005"]
      },
      {
        "id": "room_007",
        "room_type": "room_g",
        "position": { "x": 3.5, "y": 0.7 },
        "objects": [],
        "neighbors": ["room_005"]
      },
      {
        "id": "room_008",
        "room_type": "room_h",
        "position": { "x": 5.9, "y": 1.5 },
        "objects": [
          { "name": "object_013", "position": { "x": 7.4, "y": 2.3 } },
```

```
          { "name": "object_014", "position": { "x": 7.3, "y": 1.3 } },
          { "name": "object_015", "position": { "x": 7.2, "y": 0.5 } },
          { "name": "object_016", "position": { "x": 8.4, "y": 2.0 } }
        ],
        "neighbors": ["room_005"]
      },
      {
        "id": "room_009",
        "room_type": "room_i",
        "position": { "x": 6.2, "y": 6.3 },
        "objects": [
          { "name": "object_008", "position": { "x": 5.6, "y": 4.3 } },
          { "name": "object_009", "position": { "x": 6.7, "y": 3.5 } }
        ],
        "neighbors": ["room_003", "room_010", "room_011"]
      },
      {
        "id": "room_010",
        "room_type": "room_j",
        "position": { "x": 6.8, "y": 7.8 },
        "objects": [
```


4.3 方案的优点与缺点

优点：

1. 方案不依赖完整 SLAM 管线，也不要求厘米级定位精度，更多依赖视觉语义和相对运动信息即可完成房间级空间建模，因此整体工程复杂度较低，更适合原型验证和快速迭代。
2. 机器人在探索过程中即可持续采集、持续判断、持续更新图结构，不需要等任务结束后再统一离线处理，因此天然适合在线导航、交互式探索和增量式场景理解任务。
3. 相较于几何地图，这种“房间 + 物体 + 连通关系”的表达方式更贴近任务层需求，便于下游系统直接利用。

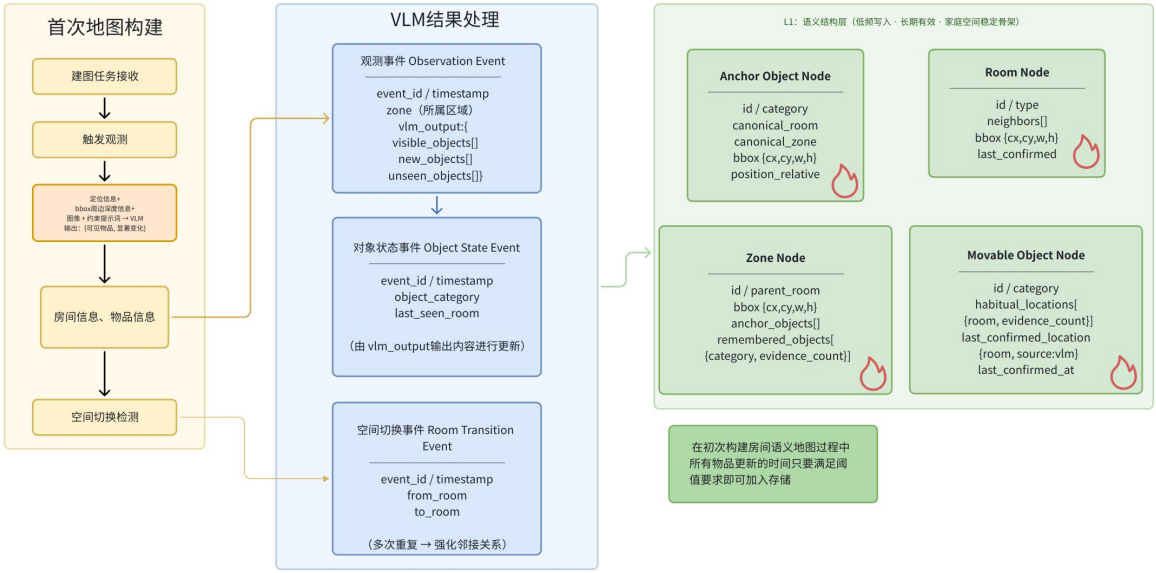
缺点：

1. 房间类型识别、物体语义判断以及区域过渡识别都依赖模型输出质量，因此模型本身的鲁棒性、提示词设计和场景覆盖能力会直接影响最终拓扑图质量。
2. 按照拓扑结构保存的地图，及时后期数据充足可大致绘制出“类似”几何分区地图的地图，但效果上可能仍没有几何地图好，在用户表现体验上存在落差。

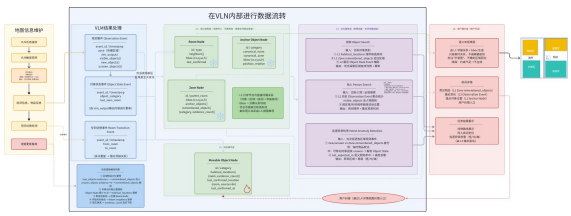
五、家庭语义记忆系统

结合传统几何美化分区方法前期使用与验证效果、第四节中对于去几何地图分区效果的初步构思，给出一个分层式的家庭语义存储系统，结构分层如下：

初次地图构建，与地图信息维护流程进行拆分，针对输入语义仅进行房间类型、区域划分、锚点物品的统计以及房间语义其他物品的初步统计，目标为快速构建初版家居场景语义，输出用户层面的家居场景图。



地图信息维护，该部分与VLN推理流程结合，伴随vln任务进行房间物品语义采集与更新，逐步搭建历史记忆，并将相关信息组装成prompt一并送入vln推理流程；



- **L1：这个家居场景长什么样**
- **L2：这个家最近发生了什么**
- **L3：怎么把前两者变成机器可用上下文**
- **L4：怎么把前几层变成人可理解界面**

5.1 L1-语义结构层

定位：家庭空间的长期稳定骨架，低频写入，长期有效。

写入来源：VLM 在线拓扑建图的聚合结果；用户主动纠错；多次观测的统计稳定结论。

5.1.1 核心数据结构

房间节点（Room Node）

代码块

```
{
  "id": "room_001",
  "type": "living_room",
  "neighbors": ["room_002", "room_003"],
  "bbox": { "cx": 3.2, "cy": 6.1, "w": 5.0, "h": 4.0 },
  "last_confirmed": "2026-03-28T10:00:00Z"
}
```

> `bbox` 来自 SLAM 或 VLM 推断，一旦由多次观测稳定确认后冻结，不随单次观测更新。

区域节点（Zone Node）

代码块

```
{
  "id": "zone_sofa_area",
  "parent_room": "room_001",
  "bbox": { "cx": 2.1, "cy": 5.8, "w": 2.5, "h": 1.8 },
  "anchor_objects": ["obj_sofa_001", "obj_coffee_table_001"],
  "remembered_objects": [
    { "category": "remote_control", "evidence_count": 8 },
    { "category": "pillow", "evidence_count": 5 }
  ]
}
```

> `bbox` 由单次观测中多个锚点物品的位置关系共同推断得出，基本可信；一旦确认后冻结，仅当区域内锚点物品集合发生变化时重新计算。

> `remembered_objects` 是该区域的持久记忆物品表，由 L2 观测事件沉淀写入：出现次数超过阈值则加入，长期未观测到则 `evidence_count` 归零后删除。L2 Observation Event 中的 `new_objects` / `unseen_objects` 均以此表为基准计算，而非相对上一次单次观测。

锚点对象节点（Anchor Object Node）-锚点对象物品包含：门、沙发、茶几、电视柜、冰箱、餐桌、床、柜子等大型不易移动家具

代码块

```
{
  "id": "obj_sofa_001",
  "category": "sofa",
  "canonical_room": "room_001",
  "canonical_zone": "zone_sofa_area",
  "bbox": { "cx": 2.1, "cy": 5.8, "w": 2.5, "h": 1.8 },
  "position_relative": "靠近西墙窗边"
}
```

> 锚点对象归属于某个区域（多个锚点对象可共属同一区域）。`position_relative` 为辅助描述，供 LLM 消费；区域几何边界由 Zone Node 的 `bbox` 承载，不在对象节点重复存储。

可移动对象记忆节点（Movable Object Memory Node）

代码块

```
{
  "id": "obj_medicine_box",
  "category": "medicine_box",
  "habitual_locations": [
    {
      "room": "room_004",
      "evidence_count": 12
    },
    {
      "room": "room_005",
      "evidence_count": 6
    }
  ],
  "last_confirmed_location": {
    "room": "room_005",
    "source": "vlm"
  },
  "last_confirmed_at": "2026-03-28T20:15:00Z"
}
```

> 可移动对象位置完全由 VLM 观测驱动。位置粒度仅到房间级，不下沉到区域（可移动对象不锚定区域）。

5.1.2 设计要点

- **L1 只存节点与直接归属关系**：对象→区域、区域→房间、房间→邻接房间；复杂语义关系（"常见于"、"习惯性出现在"、"不应出现在"等）由 L3 在查询时从 L1+L2 推导，不预先写入 L1
- **bbox 一旦确认即冻结**：区域 bbox 由单次多锚点观测推断，房间 bbox 由多次观测稳定后确认；两者均不随单次新观测更新，仅在结构发生显著变化时重算
- **节点不随单次观测失效**：只有累计多次观测触发沉淀或结构变化才更新 L1

L1 记录的是长期语义事实，不直接吞并原始观测噪声：任何结构更新都应来自 L2 的沉淀或明确初始化流程

5.2 L2-动态事件层

定位：记录高频、短生命周期的观测事件和状态变化，是"最近发生什么"的真实来源。

写入来源：VLM 每次采样的原始输出；机器人运动轨迹；传感器触发事件。

保留策略：按时间窗口保留（如最近 7 天详细，更早的压缩摘要），不无限积累。

5.2.1 核心数据结构

观测事件（Observation Event）

代码块

```
{
  "event_id": "obs_20260328_145201",
  "timestamp": "2026-03-28T14:52:01Z",
  "zone": "zone_sofa_area",
  "vlm_output": {
    "visible_objects": ["remote_control", "coffee_cup", "newspaper"],
    "new_objects": ["bottle", "hat"],
    "unseen_objects": ["medicine_box"]
  }
}
```

> 记录机器人在某区域的一次 VLM 采样结果。`visible_objects` 是本次可见物品全集；`new_objects` 是相比上次观测新出现的；`unseen_objects` 是上次可见但本次消失的。供更新机制统计对象出现频率和变化。

对象状态事件（Object State Event）

代码块

```
{
```

```
  "event_id": "obj_state_20260328_093011",
  "timestamp": "2026-03-28T09:30:11Z",
  "object_category": "medicine_box",
  "last_seen_room": "room_004"
}
```

> 记录某类可移动对象最近一次被观测到的房间。由观测事件中的 `visible_objects` 派生写入，作为更新机制更新 `habitual_locations` 的依据。

空间切换事件（Room Transition Event）

代码块

```
{
  "event_id": "trans_20260328_093500",
  "timestamp": "2026-03-28T09:35:00Z",
  "from_room": "room_001",
  "to_room": "room_002"
}
```

> 记录机器人从一个房间进入另一个房间的时刻。多次重复的切换对用于强化 L1 中两个房间的 `neighbors` 邻接关系。

5.2.2 设计要点

> L2 事件不直接暴露给上层消费，而是通过定期沉淀更新 L1：

- **区域物品记忆沉淀**：某对象在某区域的观测事件中多次出现（`evidence_count` 超过阈值）→ 加入 L1 Zone Node 的 `remembered_objects`；长期出现在 `unseen_objects` 中导致 `evidence_count` 归零 → 从 `remembered_objects` 删除
- **可移动对象位置更新**：某对象状态事件在某房间累计出现 N 次 → 更新 L1 可移动对象 `habitual_locations` 分布
- **新房间发现**：多次未匹配已有房间的观测 → 触发 L1 新节点创建
- **邻接关系强化**：空间切换事件多次重复 → 强化 L1 `neighbors` 关系
- **遗忘衰减**：L1 可移动对象长期无 L2 证据支撑 → `evidence_count` 自然下降，不直接删除节点

5.3 L3-记忆服务层

定位：不作为事实源，负责把 L1+L2 转化为不同消费者（任务执行、LLM、推理引擎）可用的形态。它首先是"翻译层"，必要时允许存在可重建的派生索引。

核心职责：按需组装，而不是预先把所有视图都写死

5.3.1 与 核心任务

```
L1（语义结构）──┐
                ├──→ L3 记忆服务层 ──→ 找物 / 找人 / 巡逻异常检测
L2（动态事件）──┘
```

L1 只存节点与直接归属关系，复杂语义关系由 L3 在查询时按需推导：

关系	推导来源	示例
`habitually_found_in`	L1 可移动对象 `habitual_locations`	药箱 → 客厅+沙发区
`not_expected_in`	常识规则 + L2 历史缺席统计	清洁剂 → 儿童房
`used_for`	VLM 描述聚合	阳台+柜子区 → 收纳清洁工具

这些关系不写入 L1，不持久化存储，查询用完即丢。

5.3.2 任务场景

找物（Object Search）

> 输入：目标对象类别（如"药箱"）
>
> 输出：优先探索的区域有序列表 + 每个区域的历史概率

```
任务："找药箱"
L3 组装输出：
  候选房间（按 habitual_locations 概率排序）：
    1. 主卫（60%，上次观测：2026-03-28 09:30）
    2. 次卫（30%，上次观测：2026-03-27 14:10）
  区域内优先探索区域：
    主卫 → zone_mirror_cabinet（remembered_objects 含 medicine_box）
  排除房间：客厅、卧室（历史概率 < 5%）
```

推导来源：L1 `habitual_locations` + L1 Zone `remembered_objects` + L2 最近对象状态事件

找人（Person Search）

输入：目标人物（如"爷爷"）或无具体目标（全局搜索）

输出：按历史活动频率排序的房间列表 + 最近观测时间

```
代码块
任务："找爷爷"
  L3 组装输出：
    高频活动区域（按 L2 历史观测统计）：
      1. 客厅沙发区（最近见于：2026-03-31 08:45）
      2. 餐厅（最近见于：2026-03-31 07:30）
    当前未覆盖房间：书房、阳台（今日尚未观测）
    建议起始房间：客厅（最近活动 + 历史最高频）
```

推导来源：L2 观测事件中的 `visible_objects` （含人物类别）按区域/时间聚合统计

巡逻异常检测（Patrol Anomaly Detection）

> 输入：当次巡逻各区域的观测事件
>
> 输出：与基线偏差显著的区域 + 异常描述

```
代码块
  巡逻结果（当次 vs 基线对比）：
    客厅茶几区：
      新增物品：bottle，hat（不在 remembered_objects 中）
      消失物品：newspaper（长期常见，本次 unseen）
      → 异常等级：低（临时物品变化）
    主卫：
      medicine_box 未见（历史常见，连续 3 次 unseen）
      → 异常等级：中（可能已被移走，触发对象状态事件更新）
    儿童房：
      检测到清洁剂（not_expected_in 规则命中）
      → 异常等级：高（安全规则违反，需告警）
```

推导来源：L2 当次观测事件（ `new_objects` / `unseen_objects` ）vs L1 Zone `remembered_objects` 基线 + `not_expected_in` 语义规则

5.4 L4-用户展示层

定位：由 L3 驱动生成，面向用户可理解性设计，不直接暴露 L1/L2 的数据结构。

核心原则：用户要的是"看懂这个家"，不是"看懂系统内部建模"。

5.4.1 三个展示视图

A. 房间总览视图（主视图）

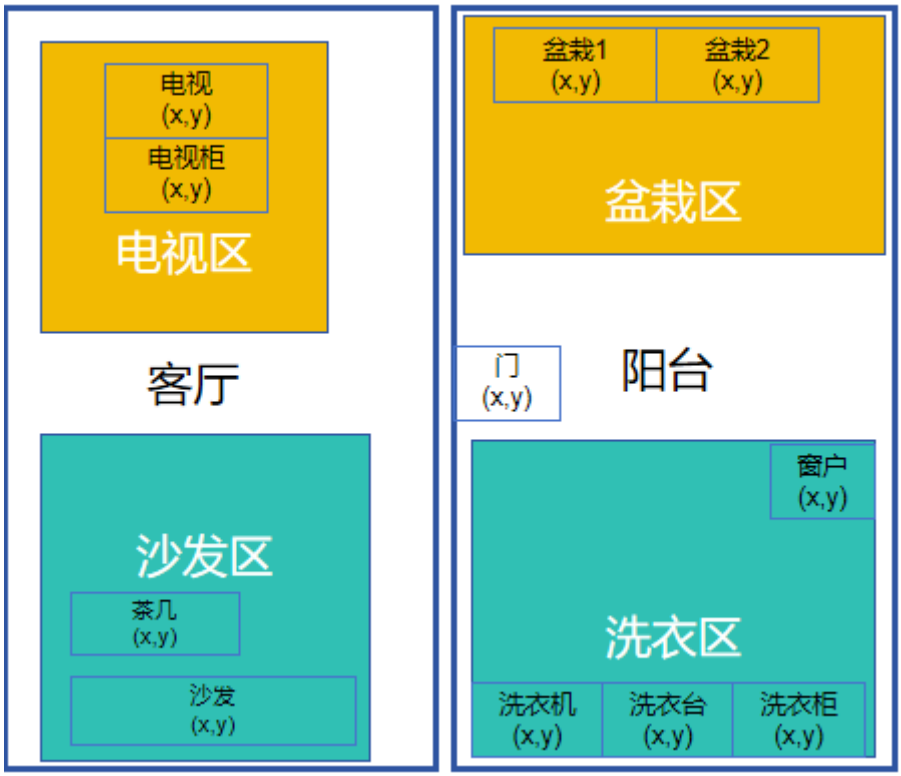
由 L1 房间节点 + 邻接关系直接驱动，展示：

- 家里有哪些房间（按 L1 房间节点）
- 房间之间如何相连（按 `adjacent_to` 关系）

- 每个房间的类型标签和主要语义
- 每个房间的区域节点显示

形式：卡片列表 + 关系连线（不是坐标图，是语义关系图）

B. 语义布局草图（地图视图）



重要：这不是几何地图，是由 L1 关系约束驱动的生成式草图。

生成输入：

- 房间邻接关系（哪些房间相连）
- 房间大小关系（L1 记录的相对面积描述）
- 关键连接点（入口、走廊）

生成原则：

- **只画相对关系，不画绝对坐标**

- **不确定区域弱化处理**，不硬画精确边界

- **允许"大致正确、布局合理"**，不追求工程精度

- 用户看到的是语义示意图，不是 SLAM 栅格图

C. 房间详情视图（点击展开）

由 L1 对象节点 + L2 最近事件驱动，展示：

- 该房间内的常见家具和高频物品（来自 L1）
- 最近观测到的状态变化（来自 L2）

- 对象置信度（系统推断 vs 用户确认）
- 用户纠错入口